

AI-Assisted Programming in ABINIT

Why fast AI code needs maps, specs, and tests

Xu He

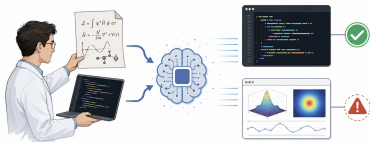
Université de Liège

Abidev2026



The tempting view

Implement DFT+U with
norm-conserving pseudopotentials in
ABINIT. AI writes code. Bingo?



The real problem

ABINIT is a maze of physics,
conventions, parallelism, and old code
paths.



B. Potentials

The wave function is expanded in the plane wave basis as:

$$|\psi_{k\mathbf{b}}\rangle = \sum_{\mathbf{i}} c_{k\mathbf{b}\mathbf{i}} |p_{k\mathbf{i}}\rangle \quad (1)$$

The overlap matrix element $S_{k\mathbf{b}\mathbf{m}}$ is defined as:

$$S_{k\mathbf{b}\mathbf{m}} = \langle l_{\mathbf{m}} | p_{k\mathbf{b}} \rangle \quad (2)$$

The on-site k -resolved density matrix in the basis of the localized orbitals can be written as:

$$\rho_{k,\mathbf{m}\mathbf{n}} = \sum_{\mathbf{b}} f_{k\mathbf{b}} \langle l_{\mathbf{m}} | \psi_{k\mathbf{b}} \rangle \langle \psi_{k\mathbf{b}} | l_{\mathbf{n}} \rangle \quad (3)$$

$$= \sum_{\mathbf{b}} f_{k\mathbf{b}} \sum_{\mathbf{i}} c_{k\mathbf{b}\mathbf{i}} \langle l_{\mathbf{m}} | p_{k\mathbf{i}} \rangle \sum_{\mathbf{j}} c_{k\mathbf{b}\mathbf{j}}^* \langle p_{k\mathbf{j}} | l_{\mathbf{n}} \rangle \quad (4)$$

$$= \sum_{\mathbf{b}} f_{k\mathbf{b}} \sum_{\mathbf{i}\mathbf{j}} c_{k\mathbf{b}\mathbf{i}} c_{k\mathbf{b}\mathbf{j}}^* S_{k\mathbf{b}\mathbf{m}} S_{k\mathbf{b}\mathbf{n}}^* \quad (5)$$

In the matrix form, it is :

$$\rho_{\mathbf{k}} = \mathbf{S}_{\mathbf{k}} \mathbf{C}_{\mathbf{k}} F_{\mathbf{k}} \mathbf{C}_{\mathbf{k}}^{\dagger} \mathbf{S}_{\mathbf{k}}^{-1} \quad (6)$$

and the density matrix $\rho = \sum_{\mathbf{k} \in BZ} \rho_{\mathbf{k}} w_{\mathbf{k}}$, with $w_{\mathbf{k}}$ the weight of the kpoint $\mathbf{k}$.

In the DFT+U formalism, the energy, and potential are functions of $\rho_{\mathbf{m}\mathbf{n}}$.

The potential is :

$$V_{\mathbf{m}\mathbf{n}} = \frac{\partial E_{DFTU}}{\partial \rho_{\mathbf{m}\mathbf{n}}} \quad (7)$$

Due to the localization of the orbitals, the potential $V_{\mathbf{b},\mathbf{m}\mathbf{n}}$ is not $\mathbf{k}$ -dependent. $V_{\mathbf{b},\mathbf{m}\mathbf{n}} = V_{\mathbf{m}\mathbf{n}}$. The $V_{\mathbf{m}\mathbf{n}}$ follows the standard DFT+U procedure, can be evaluated with the subroutines for PAW.

This can be projected from the localized basis set to the planewave basis set, and then added to the DFT potential:

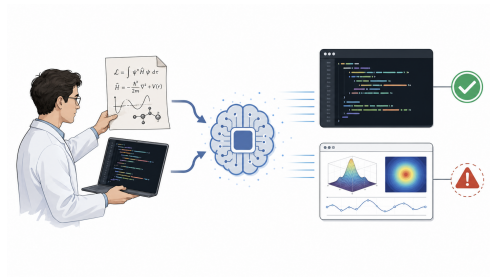
Input to AI

- scientific draft with equations;
- ABINIT source code;
- prompts and discussion.

Constraint

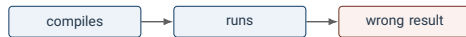
Handcrafted code is forbidden:
use the AI-assisted workflow.

- AI researched the ABINIT codebase.
- It identified routines and files to modify.
- It produced a first implementation in a few hours.
- The code compiled and could run a calculation.



But

The result was wrong.



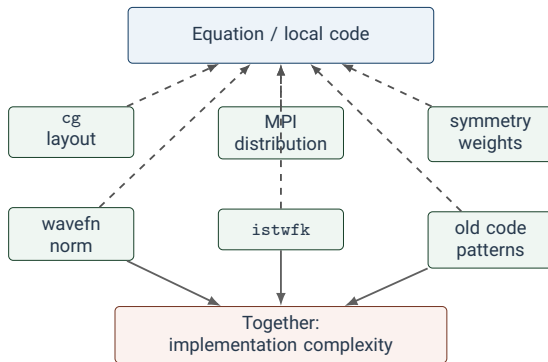
$$n_i = \sum_{n,k} f_{nk} |\langle \psi_{nk} | \phi_i \rangle|^2$$

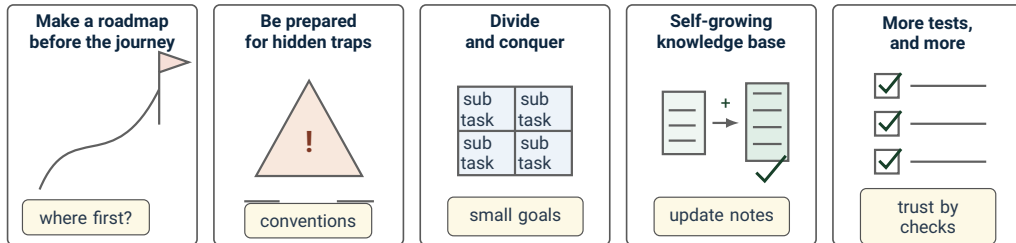
What went wrong

The formula was clear, but the implementation depended on conventions outside the local code snippet.

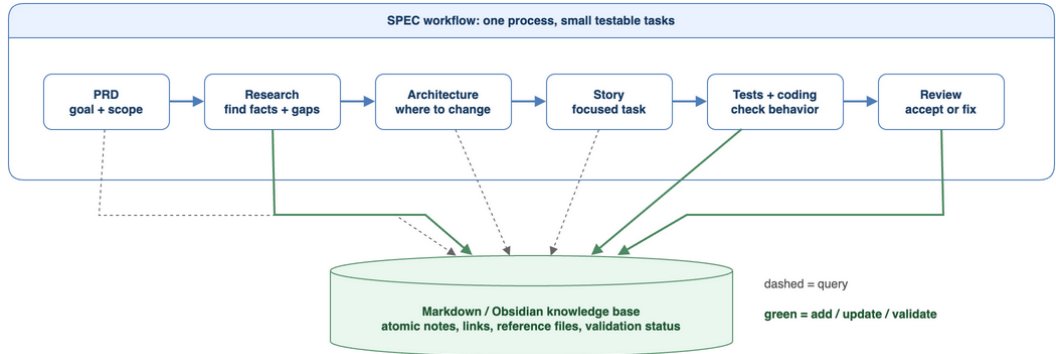
- cg layout, istwfk, MPI distribution
- k-point weights, symmetry, normalization
- old code patterns and developer experience

Together, this is too much context for one big AI task.





The workflow makes these habits explicit before implementation starts.



- PRD: what problem are we solving?
- Research: what must be understood first?
- Architecture: what decisions matter?
- Story: what small unit can be implemented?
- Tests/review: how do we accept or reject it?

Story 002: Implement `compute_occmatrix_deriv_nc`

User Story

As a developer,
I want occupation matrix derivatives $\frac{\partial n_{ij}}{\partial U_i}$ and $\frac{\partial n_{ij}}{\partial U_j}$ computed,
So that DFT+U forces and stress can be calculated.

Acceptance Criteria

- ☐ Given wavefunctions and PAO projections, when `compute_occmatrix_deriv_nc` is called, then $\frac{\partial n_{ij}}{\partial U_i}$ is computed for all atoms
- ☐ Given wavefunctions and PAO projections, when `compute_occmatrix_deriv_nc` is called, then $\frac{\partial n_{ij}}{\partial U_j}$ is computed
- ☐ Given NIO DFT+U system, when $\frac{\partial n_{ij}}{\partial U_i}$ is computed, then it matches finite-difference of $n_{ij}(R_i + \delta R) - n_{ij}(R_i)$

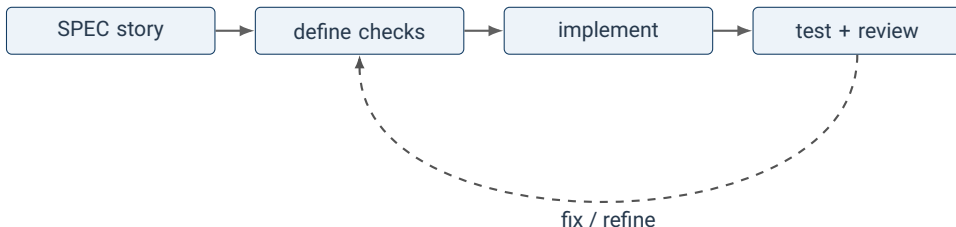
Tasks

Phase 1: Research

- ☐ Search for `compute_occmatrix_nc` patterns in `n_nc_dftu.F90`
- ☐ Explore PAW occupation matrix derivatives in `n_paw_occupancies.F90` (option=3)
- ☐ Check Hellmann-Feynman approximation: $\frac{\partial E_{\text{total}}}{\partial U_i} \approx \langle \frac{\partial \psi}{\partial U_i} | \psi \rangle$
- ☐ Document formula: $\frac{\partial n_{ij}}{\partial U_i} = \sum_{a,b} w_{ab} f_{ab} [\langle \frac{\partial \psi_a}{\partial U_i} | \psi_{ab} \rangle \langle \phi_j | \psi_{ab} \rangle + \langle \phi_i | \psi_{ab} \rangle \langle \frac{\partial \psi_b}{\partial U_i} | \psi_{ab} \rangle]$

Phase 2: Tests (TDD)

- ☐ TEST-001: Write test for $\frac{\partial n_{ij}}{\partial U_i}$ — compare with finite differences



- Checks define the target before code is written.
- AI implementation is accepted or rejected by evidence.

ABINIT difficulty

Unit tests are hard to write; full end-to-end tests are expensive. We need practical checks at the right scale.

Atomic note contents

- topic name and tags;
- confidence and validation status;
- source files and tests used to validate;
- links to related SPEC documents;
- updates from review and implementation.

abinit_wavefunction_storage

Properties

confidence	0.98
created	03/03/2026 🔗
last_validated	03/03/2026 🔗
validation_pro...	Source code analysis of vtorho, m_mikrho, m_bandfft_kpt, m_nc_dftu; confirmed by MPI parallelization tests (npkpt, npband, npfft, combined)
project	abinit
status	validated
validated_by	automatic
tags	wavefunction X storage X parallelization X cg-array X para_l_kgb X bandfft X
+ Add property	

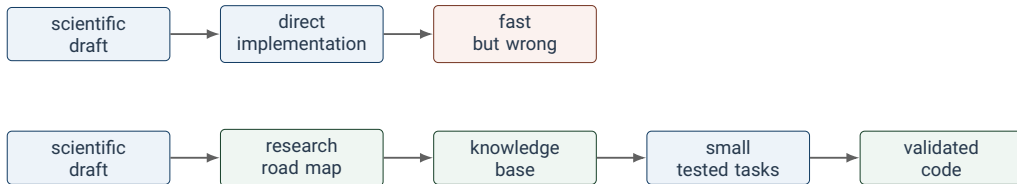
Overview

Wave functions in ABINIT are stored in the `cg_array` (plane wave coefficients). The layout of `cg` depends critically on the parallelization mode: `para_l_kgb=0` uses classic k-point/band distribution, while `para_l_kgb=1` (bandfft scheme) additionally distributes G-vectors across procs. Understanding the distinction is essential for any code that accesses `cg` directly.

Two Parallelization Modes

Mode A: `para_l_kgb=0` (classic)

Each proc owns a subset of k-point, band, spin) triads. All proc k-G-vectors for an



Change in workflow

The scientific draft should not go directly to a full implementation. It first becomes a road map, validated notes, small SPEC stories, and tests.

The workflow helps AI, but it also helps human developers.

- Fewer hidden assumptions.
- Better onboarding for difficult code paths.
- Clearer review of scientific changes.
- Safer incremental development.
- Reusable implementation knowledge.

1. AI is useful, but not automatic.
2. ABINIT-specific context is the main challenge.
3. SPEC workflow makes large tasks small enough to reason about.
4. Knowledge notes make hidden conventions explicit.
5. Tests and review turn AI output into usable scientific code.

Thank You

Questions?